

Algorithmique et programmation impérative

Chapitre 2 : Types, variables et expressions

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 2. Types, variables et expressions

- 1 Valeurs et types
- 2 Variables
- 3 Opérateurs et expressions
- 4 Un mot sur les fonctions
- 5 Erreurs fréquentes
- 6 Exercices interactifs
- 7 Glossaire

Un peu de mentalisme

Avant d'aborder ce chapitre, je vous propose une petite séance de mentalisme.



Source de l'image : <https://www.artesine.fr>

Valeurs et types

Valeurs et types

Une **valeur** est un des éléments de base d'un programme, comme un nombre ou un mot. Exemples : 1, 2.5 ou 'Bonjour'

Toute valeur possède un **type** particulier.

- 1 et 2 sont des entiers (**integer**) : `int`
- 1.4 et 2.0 sont des réels (nombres à virgule flottante, **floating-point**) : `float`
- 'Bonjour' est une chaîne de caractères (**string**), identifiable (par vous et l'interpréteur Python) grâce aux apostrophes : `str`

(Un type de valeurs $\simeq$ **classe** de valeurs d'un même type. Il y a beaucoup de types différents et vous apprendrez à créer les vôtres !)

Valeurs et types

L'interpréteur peut vous donner le **type** d'une valeur :

```
>>> type(4)
<class 'int'>
>>> type('Bonjour')
<class 'str'>
>>> type(1.2)
<class 'float'>
```

Remarque : `type` est une **fonction** prédéfinie de Python. Un appel à une fonction est visible par l'utilisation des parenthèses. On reviendra sur la notion de fonction plus loin dans ce chapitre et le suivant.

Valeurs et types

Il y a des différences syntaxiques pour que Python reconnaisse les types :

- le point détermine un `float`
- les apostrophes (ou les guillemets) déterminent un `str`

```
>>> type(2)
<class 'int'>
```

```
>>> type(2.)
<class 'float'>
```

```
>>> type('2')
<class 'str'>
```

```
>>> type("2")
<class 'str'>
```

Variables

Variables et assignation

Une **variable** est un nom (symbolique) qui permet de faire référence à une valeur (et de la stocker en mémoire).

Une **assignation** est une instruction qui permet de créer une nouvelle variable et de lui attribuer une valeur.

Syntaxe d'une assignation : `nom = valeur`

```
>>> i = 12  
>>> message = 'La valeur de PI est'  
>>> pi = 3.14159265
```

Une fois assignée, on peut retrouver et utiliser la valeur stockée en mémoire en utilisant le nom de la variable.

Variables et assignation

En Python, on peut **afficher** le contenu d'une (ou plusieurs) variable(s)

- à l'aide de la fonction `print` (mode script et interactif)
- en donnant simplement le nom d'une variable (uniquement en mode interactif)

```
>>> print(i)
```

```
12
```

```
>>> print(message, pi)
```

```
La valeur de PI est 3.14159265
```

```
>>> i
```

```
12
```

Remarque : en mode script, la fonction `print` est nécessaire pour afficher une variable (testez-le!).

Variables et assignation

Tant qu'une variable n'est pas assignée, elle n'existe pas et on ne peut donc pas afficher sa valeur.

```
>>> print(j)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'j' is not defined
>>> j = 18
>>> print(j)
18
```

Remarque : le message d'erreur ci-dessus est pour le moment un peu cryptique mais la dernière ligne de celui-ci est explicite :

```
NameError: name 'j' is not defined
```

Variables et assignation

Une **assignation** permet également de mettre à jour la valeur d'une variable déjà créée.

```
>>> print(i)
```

```
12
```

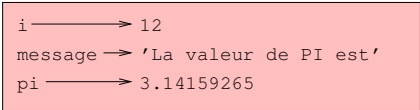
```
>>> i = 5
```

```
>>> print(i)
```

```
5
```

Visualisation de l'état des variables

On peut représenter les variables par un **diagramme d'état**.



```
i —————> 12  
message —> 'La valeur de PI est '  
pi —————> 3.14159265
```

Vous pouvez également utiliser **Python Tutor** pour suivre pas à pas la “vie” de vos variables en mémoire.

<https://pythontutor.com/visualize.html>

Ces méthodes visuelles seront bien utiles pour comprendre certaines subtilités !

Visualisation de l'état des variables

Démonstration : Python Tutor

Python 3.6
([known limitations](#))

```
1 i = 42
2 x = 2.6175
→ 3 print(i)
→ 4 i = 9
5 msg = 'hello'
6 print(i, x, msg)
```

[Edit this code](#)

→ line that just executed

→ next line to execute

<< First

< Prev

Next >

Last >>

Step 4 of 6

Print output (drag lower right corner to resize)

42

Frames

Objects

Global frame

i	42
x	2.6175

Type d'une variable

Le type d'une variable est le type de la valeur qu'elle réfère.

```
>>> type(i)
<class 'int'>
>>> type(message)
<class 'str'>
>>> type(pi)
<class 'float'>
```

Précisions à propos de la syntaxe d'une assignation

Rappel de la syntaxe d'une assignation :

```
nom = valeur
```

A gauche (nom)

Vous avez (presque) toute liberté pour nommer votre variable

A droite (valeur)

Vous pouvez coder tout ce que vous voulez tant que Python peut en déduire une valeur

Précisions à propos de la syntaxe d'une assignation

Noms des variables

Le nom d'une variable

- Contraintes :
 - ▶ ne peut contenir que des lettres (minuscules et majuscules), des chiffres et le caractère `_` (underscore)
 - ▶ doit commencer par une lettre ou le caractère `_`
- Conventions :
 - ▶ devrait avoir du sens (dire ce qu'elle représente)
 - ▶ devrait commencer par une lettre minuscule

Attention à la casse : `pi` est différent de `Pi` ou de `PI`

Exemples de noms valides : `x1`, `maVariable`, `mon_entier`

Exemples de noms invalides : `2be`, `x@`, `class`

Pourquoi `class` n'est-il pas valide ?

Précisions à propos de la syntaxe d'une assignation

Mots-clef

On ne peut pas utiliser `class` pour une variable car c'est un mot-clef (mot réservé) du langage Python.

Python 3 possède 33 mots-clef :

<code>False</code>	<code>class</code>	<code>finally</code>	<code>is</code>	<code>return</code>
<code>None</code>	<code>continue</code>	<code>for</code>	<code>lambda</code>	<code>try</code>
<code>True</code>	<code>def</code>	<code>from</code>	<code>nonlocal</code>	<code>while</code>
<code>and</code>	<code>del</code>	<code>global</code>	<code>not</code>	<code>with</code>
<code>as</code>	<code>elif</code>	<code>if</code>	<code>or</code>	<code>yield</code>
<code>assert</code>	<code>else</code>	<code>import</code>	<code>pass</code>	
<code>break</code>	<code>except</code>	<code>in</code>	<code>raise</code>	

Si l'interpréteur se plaint du nom d'une de vos variables, consultez cette liste !

Précisions à propos de la syntaxe d'une assignation

Comment définir une valeur ?

Une valeur (à droite dans l'assignation) peut être obtenue

- en la donnant directement (ce qu'on a fait jusqu'ici, mais si on se limitait à ça, nos programmes ne seraient pas très impressionnants !)
- via le nom d'une variable
 - ▶ cela peut être une autre variable déjà définie
 - ▶ ou la variable elle même si elle a déjà été définie auparavant
- via le résultat d'une expression arithmétique
- et via d'autres mécanismes que vous découvrirez petit à petit comme le résultat d'une fonction qui peut cacher un algorithme aussi complexe que possible

Il est possible de combiner tout cela tant qu'au final Python peut en déduire une valeur !

Précisions à propos de la syntaxe d'une assignation

Comment définir une valeur ?

Exemples

```
>>> a = 13
>>> b = a
>>> c = 3 + 2
>>> d = b + c
>>> a = a + 1
>>> print(a, b, c, d)
14 13 5 18
```

Remarque : notez que même si `b` à été initialisé avec la valeur de `a`, ces deux variables n'ont plus la même valeur à la fin de ces instructions. Comprenez-vous pourquoi ?

Votre premier algorithme

Problème

Vous avez deux variables a et b en entrées. Comment faire pour échanger leurs valeurs ?

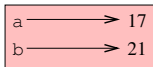
```
>>> a = 17
>>> b = 21
>>> ???
>>> print(a)
21
>>> print(b)
17
```

(Utilisation de PythonTutor pour tester les solutions proposées par les étudiants)

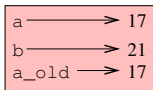
Votre premier algorithme

Solution : utiliser une troisième variable temporaire pour ne pas perdre de valeur

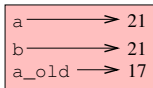
```
>>> a = 17
>>> b = 21
```



```
>>> a_old = a
```



```
>>> a = b
```

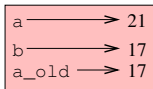


```
>>> b = a_old
>>> print(a)
```

21

```
>>> print(b)
```

17



Opérateurs et expressions

Opérateurs et expressions

Python supporte toute une série d'opérateurs arithmétiques pour créer des expressions. On a déjà vu l'addition. Le symbole `*` est utilisé pour la multiplication.

```
>>> pi = 3.14159265
>>> rayon = 10
>>> circonference = 2 * pi * rayon
>>> aire = pi * rayon * rayon
>>> print(circonference)
```

62.831853

```
>>> print(aire)
```

314.59265

Opérateurs

Les **opérateurs** sont des symboles spéciaux qui représentent des opérateurs arithmétiques comme l'addition ou la division. Les **opérandes** sont des valeurs de type `int` ou `float`. Comme pour l'assignation, cela ne doit pas forcément être une valeur explicitement donnée, du moment que Python peut en déduire une valeur.

- $+$ → addition (`20 + 32`)
- $-$ → soustraction (`hour - 1`)
- $*$ → multiplication (`hour * 60 + minute`)
- $/$ → division (`minute / 60`)
- $**$ → exposant (`5 ** 2`)
- $//$ → division entière (`7 // 2`)
- $\%$ → modulo (`7 % 2`) : reste de la division entière

Les parenthèses peuvent également être utilisées :

`(5 + 9) * (15 - 7)`

Division de deux entiers et Division entière

Division de deux entiers : retourne une valeur de type `float`

```
>>> 7 / 2
3.5
>>> 4 / 2
2.0
```

Division entière : retourne une valeur de type `int` qui est la partie entière du résultat.

```
>>> 7 // 2
3
>>> 7 % 2
1
```

Remarque : comportement très différent en Python 2 (voir note sur moodle)

Expressions

Une **expression** est une combinaison de valeurs, variables et opérateurs.

Exemples :

- 17
- x
- $x + 17$

En mode interactif, l'interpréteur **évalue** l'expression et affiche le résultat.

```
>>> 1 + 1
```

2

Par contre, dans un script, une expression est évaluée mais le résultat n'est pas affiché : on peut croire qu'il ne se passe rien si on ne demande pas d'afficher le résultat !

Règles de précedence

L'ordre de précedence des opérateurs est le même qu'en mathématiques :

- ❶ parenthèses (donc $2 * (3 - 1)$ donne 4)
- ❷ exposant (donc $2 ** 1 + 1$ donne 3 et $3 * 1 ** 3$ donne 3 et pas 27)
- ❸ multiplication et division (donc $2 * 3 - 1$ donne 5)
- ❹ addition et soustraction

Les opérateurs ayant la même précedence sont évalués de gauche à droite.

Exemple : $d / 2 * \text{pi}$: la division est effectuée en premier, et le résultat est donc $\frac{d \cdot \pi}{2}$.

Pour obtenir $\frac{d}{2\pi}$, il faut écrire $d / (2 * \text{pi})$

Opérations sur les chaînes de caractères

On ne peut pas utiliser tous les opérateurs mathématiques sur des opérandes de type `str`.

Opérateurs acceptés pour les chaînes

- l'opérateur `+` permet de **concaténer** deux `str` (attacher le deuxième `str` au premier)
- l'opérateur `*` utilisé sur un `str` et un `int` permet de **répéter** le `str`

```
>>> first = 'chat'
>>> second = 'eau'
>>> together = first + second
>>> print(together)
chateau
```

```
>>> 'ha' * 3
'hahaha'
```

Un mot sur les fonctions

Réutiliser un calcul

Soit le code suivant :

```
>>> pi = 3.14159265
>>> rayon = 10
>>> circonference = 2 * pi * rayon
>>> print(circonference)
```

62.831853

```
>>> rayon = 2
>>> print(rayon)
```

2

D'après vous, quelle valeur sera affichée si on affiche à nouveau `circonference` ? La circonférence d'un cercle de rayon 10 ($\simeq 62.83$) ou celle d'un cercle de rayon 2 ($\simeq 12.57$) ?

Réutiliser un calcul

Bien que le rayon ait été modifié, la circonférence n'est pas mise à jour automatiquement. Il faut recalculer la nouvelle circonférence.

```
>>> print(circonference)
```

```
62.831853
```

```
>>> circonference = 2 * pi * rayon
```

```
>>> print(circonference)
```

```
12.5663706
```

Ce comportement est normal au vu du mécanisme de l'assignation :

⇒ [démonstration via Python Tutor](#)

Réutiliser un calcul

Devoir « copier / coller » le calcul de la circonférence n'est pas très élégant ! Heureusement, nous pouvons plutôt définir `circonference` comme une **fonction** qui prend en paramètre le rayon et cache le code du calcul de la circonférence.

```
pi = 3.14159265

def circonference(rayon):
    resultat = 2 * pi * rayon
    return resultat

circl = circonference(10)
circ2 = circonference(2)

print('Circonference: cercle1 =', circl, ', cercle2 =', circ2)
```

⇒ **exécution de ce script et démonstration via Python Tutor**

Définir une fonction

```
def circonference(rayon):  
    resultat = 2 * pi * rayon  
    return resultat
```

Remarques :

- `def` est un mot-clef qui introduit la définition d'une fonction
- il est suivi du nom de la fonction (même contraintes que pour les noms de variables) ; puis d'une liste d'arguments (séparés par des virgules) entre parenthèses (ici, il n'y en a qu'un) ; et enfin du symbole « : »
- le code de la fonction est **indenté** (attention : toujours le même nombre d'espaces)
- on retourne la valeur souhaitée via `return`

Les fonctions

On reconnaît l'appel à une fonction via les parenthèses et, quand elles retournent une valeur, elles peuvent être utilisées dans le code où une telle valeur est attendue, comme dans une assignation, une expression ou un argument d'une autre fonction.

Exemples :

```
x = circonference(10)
y = circonference(10) + circonference(5)
z = circonference(circonference(10))
```

Les fonctions

Il y a toute une série de fonctions déjà prédéfinies en Python, comme `type` et `print`. D'autres sont disponibles dans des « modules » qu'il faut importer.

Ces modules contiennent parfois aussi des variables. C'est le cas pour π que nous ne devons donc plus redéfinir nous même dorénavant.

```
>>> import math
>>> math.cos(0)
1.0
>>> math.pi
3.141592653589793
>>> math.cos(math.pi)
-1.0
```

Le chapitre suivant revient en détails sur les fonctions.

Erreurs fréquentes

Erreurs fréquentes

- utiliser un mot-clef comme nom de variable (par ex. `class`)
- mettre un espace dans le nom d'une variable (par ex. `bad name`)
- utiliser une variable avant de l'avoir créée (rappel : pour créer une variable, la placer à gauche d'une assignation)

```
>>> principal = 257.50
>>> interet = principal * taux
Traceback (most recent call last):
File "<pyshell\#88>", line 1, in <module>
interet = principal * taux
NameError: name 'taux' is not defined
```

- ne pas respecter la “casse” de caractères : par ex., `pi` est différent de `Pi`
- erreur sémantique sur l'ordre des opérations :
par ex., écrire `1.0 / 2.0 * pi` pour $\frac{1}{2\pi}$

Exercices interactifs

Exercice

On a vu que $n = 42$ est une instruction valide.

Qu'en est-il de $42 = n$?

Exercice

On a vu que $n = 42$ est une instruction valide.

Qu'en est-il de $42 = n$?

Exercice

Que se passe-t-il avec $x = y = 1$?

Exercice

On a vu que $n = 42$ est une instruction valide.

Qu'en est-il de $42 = n$?

Exercice

Que se passe-t-il avec $x = y = 1$?

Exercice

En math, on peut multiplier x et y de la façon suivante : xy . Que se passe-t-il si on essaye ça en Python ?

Exercice

Supposons que nous exécutons les instructions suivantes.

```
largeur = 17  
hauteur = 12  
letter = 'x'
```

Pour chacune des expressions suivantes, quel est la **valeur** de l'expression et son **type** ?

- `largeur / 2`
- `largeur // 2`
- `hauteur / 3`
- `1 + 2 * 5`
- `letter * 5`

Exercice

Il est 19h47 et vous vous apprêtez à regarder la version longue de *La Communauté de l'Anneau* qui dure 3 heures et 48 minutes. A quelle heure votre film sera-t-il terminé ?

Ecrivez un script qui répond à cette question.

Glossaire

Glossaire

valeur : unité élémentaire de données, comme un nombre ou une chaîne de caractères, qu'un programme manipule.

type : catégorie de valeurs. Les types vus jusqu'à présent : entiers (`int`), réels (`float`), et chaînes de caractères (`str`).

variable : nom qui réfère à une valeur.

diagramme d'état : représentation graphique d'un ensemble de variables et des valeurs qu'elles réfèrent.

assignation : instruction qui assigne une valeur à une variable (`variable = valeur`)

opérateur : symbole spécial qui représente une opération simple comme l'addition, la multiplication ou la concaténation de chaînes de caractères.

opérande : une des valeurs sur lesquelles un opérateur s'applique.

division entière : opération qui divise deux nombres entiers et retourne un entier (uniquement la partie entière du résultat).

Glossaire

expression : combinaison de variables, d'opérateurs et de valeurs et dont le résultat est une valeur.

évaluer : simplifier une expression en appliquant les opérations dans le but d'obtenir la valeur résultat.

règles de précedence : ensemble de règles définissant l'ordre dans lequel les expressions impliquant plusieurs opérateurs et opérandes sont évaluées.

concaténer : joindre bout à bout des chaînes de caractères.

mot-clef : mot réservé et utilisé par le langage Python pour parser un programme ; on ne peut pas utiliser les mots-clef comme nom de variables.